

RankCal: Counterfactual Influence Quantization for Action-Conditioned Latent World-Model Planning

Method. RankCal (Counterfactual Influence Quantization)

Motivation

Phase 1 identifies a structural bottleneck: the effects of post-training quantization (PTQ) in world-model planning change with bit-width, granularity, module, and planning horizon. The planning objective can therefore look healthy while task success drops. Here, the planner scores imagined action sequences with an action-conditioned latent dynamics model. A local perturbation can reorder candidates, change the selected action, and change later latent states during receding-horizon closed-loop execution. A tensor-MSE heuristic measures reconstruction error, but the downstream object is an ordering rather than a reconstruction target.

Recent work makes this gap operationally testable without changing the planner. The full-text-backed ‘arxiv:2608.24855v1’ studies latent trajectory generation and rollout reranking, the connector-verified but abstract-only ‘arxiv:2607.28405v1’ exposes deployment-mismatched WAM PTQ concerns, and the full-text-backed ‘arxiv:2602.02110v1’ supplies the closest systematic world-model quantization sweep. Public visual continuous-control tooling and local inference make a site-only counterfactual probe practical. Explicit horizon-wise planning and backend-level byte accounting turn the question into a bounded decision-level audit. Actual mixed weight/activation (W/A) backend support and measured ‘PeakBytes’ remain implementation conditions rather than assumptions.

‘arxiv:2602.02110v1’ is the closest empirical anchor. It systematically swept weight-only and joint W/A PTQ across methods, granularity, modules, and planning horizons, and reported a mismatch between planning objectives and task success. It did not define a frozen-model, site-only counterfactual candidate-order influence estimand, or use that profile to solve a measured ‘PeakBytes’-constrained mixed W/A allocation. Its measurement axes and aggregate outcomes did not include a one-site intervention connecting candidate-order discordance in imagined rollouts to the bit allocator.

‘arxiv:2608.24855v1’ replaced repeated CEM search with rectified-flow latent trajectory generation, inverse-dynamics decoding, and frozen-LeWM rollout reranking. It did not alter PTQ calibration or bit allocation for a frozen latent planner using site-specific ranking influence from multi-step quantized rollouts. Its contribution redesigns how plans are generated rather than measuring and allocating numerical precision according to decision-level sensitivity.

‘arxiv:2603.12231v3’ used a curvature regularizer to straighten local latent trajectories and improve the conditioning of latent distance and planning objectives. It did not estimate site-only counterfactual ranking influence under quantized action-conditioned rollouts or use that estimate for mixed-precision PTQ. It changes representation geometry during training, whereas RankCal needs a post-training intervention and allocation mechanism for a fixed planner and latent dynamics model.

The connector-verified but abstract-only ‘arxiv:2607.28405v1’ proposed coordinate-compatible activation evidence, joint video-action saliency, and fixed-intervention auditing for World Action Model PTQ. It did not target imagined action-sequence latent-dynamics planner rankings or derive weight/activation bits from a site-only rollout-ranking influence profile. Its branch follows iterative joint video/action denoising and WAM coupling, so its intervention target differs from the closed-loop consequences of imagined action-sequence planning.

Closing the gap would make mixed-precision PTQ a decision-preservation tool for imagined action-sequence planners. It would expose when two allocations with similar nominal bit-width have different candidate-order and closed-loop consequences. It would give the broad sweep in ‘arxiv:2602.02110v1’ a site-level diagnostic interpretation and a deployable allocator interface, while remaining orthogonal to the planner redesign in ‘arxiv:2608.24855v1’ and separate from ‘arxiv:2607.28405v1’'s WAM joint video/action denoising branch.

Method

M1__background

Set up the frozen imagined-rollout probe that the contribution is built on.

1. Start with the frozen FP16 (16-bit floating-point) action-conditioned latent world model. Create a ‘site_manifest’ with one stable site ID for every quantizable weight/activation (W/A) location, its module or operator path, tensor shape and layout, calibration-data source, allowed bit pairs, and execution status. Freeze the planner inputs, candidate action-sequence pool with stable candidate IDs, latent starting states, rollout horizons, receding-horizon schedule, the scalar score already used by the planner, and its tie rule. The evaluator must output that score and the latent-state trace for each candidate and horizon. Select and record a concrete inference backend, meaning the runtime that executes kernels, and its support manifest: mark a pair ‘backend_native’ only when the

deployed kernel and storage layout really execute it; mark PyTorch ‘torch.ao’ fake quantization/dequantization as ‘emulation_only’ when it only imitates the arithmetic. **【author decision: name the target backend and supported W/A kernel/layout matrix, and decide which unsupported pairs are emulation-only】** For each site and allowed bit pair, quantize only that site and keep every other site in FP16. Replay the same inputs, order, declared random seeds, calibration procedure, and tie rule. Record quantizer parameters, runtime provenance, and execution mode. Return aligned FP16 and one-site outputs keyed by site, bit pair, horizon, and candidate ID; do not combine native and emulated rows. *Roll out each candidate action sequence through the frozen action-conditioned latent dynamics.*

$$z_{t+1} = F_\theta(z_t, a_t), \quad \tau(a_{0:H-1}) = (z_0, a_0, z_1, \dots, a_{H-1}, z_H) \quad (1)$$

Why: This isolates a site-level perturbation in the action-conditioned latent-dynamics branch and prevents planner redesign or WAM denoising changes from being mistaken for quantization effects.

M2_rankcal

Measure site-by-bit rollout-ranking influence and convert it into the memory-constrained mixed W/A assignment.

- For every quantization site and allowed bit pair, replay the frozen probe with exactly the same candidate pool, latent starting states, declared horizons, and random seeds. Define the comparison set as every unordered pair of different candidates in that fixed pool. For each pair, compare whether the difference between the two existing planner scores is positive, zero, or negative in FP16 and in the one-site run, following E2; store the resulting candidate-order agreement and its disagreement-derived influence for each horizon. A zero score difference stays a tie; use the tie rule fixed in the frozen-probe step only when the planner chooses an action, and never add random jitter or a new score. If the rollout is stochastic, keep the score for each declared random seed and apply one fixed aggregation rule to every row. **【author decision: choose single-trace or seed-set aggregation and fix whether aggregation occurs before or after pairwise comparison】** Store one row per site, bit pair, horizon, and execution mode with the agreement, $I_g(b)$, score provenance, calibration revision, and backend status. Keep real-backend and ‘torch.ao’ emulation rows separate; only supported rows can be used by step 3 (Use the site-by-bit influence table...). *Measure pairwise candidate-order agreement after quantizing only site g with bit pair b , then convert disagreement into influence.*

$$A_g(b) = \frac{1}{|\mathcal{P}|} \sum_{(i,j) \in \mathcal{P}} \mathbf{1} \left[\text{sgn}(s_i - s_j) = \text{sgn}(s_i^{(g,b)} - s_j^{(g,b)}) \right], \quad I_g(b) = 1 - A_g(b) \quad (2)$$

Why: The ranking estimand closes the gap between tensor reconstruction error and the planner’s decision-level sensitivity, including the downstream consequence of an altered selected action.

- Use the site-by-bit influence table from step 2 (For every quantization site *and*...), the allowed bit-pair list, the backend support manifest, the fixed target histogram, and the stated ‘PeakBytes’ ceiling. Give every quantizable site one weight/activation bit pair. Define the histogram before searching and record whether it counts sites or tensor elements. **【author decision: choose site-count or element-weighted histogram semantics and fix exact byte equality or a predeclared matching tolerance before seeing results】** For each complete assignment, build the entire post-training quantization (PTQ) checkpoint in the target runtime and measure end-to-end ‘PeakBytes’ with one declared harness, from checkpoint loading through the full planner trace, including model storage, quantizer metadata, runtime workspaces, planner and candidate state, latent rollout buffers, and temporary allocations. **【author decision: fix the end-to-end measurement boundary and allocator counter, including whether the reported value is device allocated or reserved bytes】** Do not replace that measurement with an arithmetic estimate or with memory reported by a ‘torch.ao’ emulation. Search complete assignments, keep only those that meet the fixed histogram and measured ceiling, and choose the one with the smallest aggregate site influence defined in E3. Record a per-site assignment manifest with support mode, measured peak, and histogram. Build the uniform-bit and local tensor-reconstruction comparators with the same calibration data, runtime, histogram, and measurement harness. **【author decision: define the local reconstruction scalar and its weight/activation normalization, and define how a uniform comparator is represented when exact histogram or measured-PeakBytes matching is unavailable】** Report RankCal as matched only when the compared assignments have the same histogram and the same predeclared measured-byte rule; otherwise label the comparison unmatched. *Choose the mixed W/A assignment with the smallest aggregate ranking influence under a measured peak-memory budget and fixed bit histogram.*

$$\mathbf{b}^* = \arg \min_{\mathbf{b}} \sum_{g \in \mathcal{G}} I_g(b_g) \quad \text{s.t.} \quad \text{PeakBytes}(\mathbf{b}) \leq B, \text{Hist}(\mathbf{b}) = \mathbf{h} \quad (3)$$

Why: This converts the diagnostic profile into a deployable mixed W/A PTQ assignment without changing the latent planner or replacing measured memory with a nominal byte assumption.

M3_validation

Evaluate candidate ordering, first-action agreement, and closed-loop task success for the unchanged planner.

4. Apply the selected assignment to every quantizable site and run the real target-backend checkpoint through the unchanged planner. Use a versioned ‘evaluation_manifest’ listing task and episode IDs, initial states, candidate pool, declared horizons, replanning interval, stopping rule, random seeds, and the existing success predicate; replay it identically for FP16, RankCal, the uniform-bit and local-reconstruction comparators, and the permutation control. **【author decision: name the evaluation manifest and all of its task, seed, horizon, and success fields before any comparison】** At each replanning point, record the full candidate order, pairwise order agreement relative to FP16 using E2, the first action selected after the tie rule fixed in the frozen-probe step, and the existing closed-loop task-success result. Aggregate by horizon and episode without changing the planner. Record the fixed histogram, end-to-end ‘PeakBytes’, native or emulation status, and byte-matching stratum with each result. The final evaluation must use a complete all-sites assignment, not only the one-site probes. Within the same all-sites evaluation, use the predeclared representative high/low site pairs and build complete matched variants: force the high site alone, the low site alone, or both sites jointly, while assigning every other site with the same declared completion rule. Measure every variant with the same candidate pool, histogram, measured ‘PeakBytes’ target, planner schedule, receding-horizon convention, and tie rule, and use the same E2-derived candidate-order discordance for the single-site and joint rows. **【author decision: fix the pair-selection rule, horizon and bit stratum, tie-break, and full-assignment completion before observing outcomes】** If an exact histogram-and-byte match cannot be constructed, mark the pair diagnostic ‘unavailable’ and do not compare those rows. Keep this diagnostic as a tactical check inside step 4 (Apply the selected assignment *to...*) : do not turn it into a new summary target (estimand), add a step ID, claim joint optimality, or change the pre-existing permutation-based negative-control direction; a discrepancy can only limit how strongly the site-wise allocation signal is interpreted.

Why: The object to preserve is the selected action and its later consequences, so the evaluation stays in latent-dynamics planning rather than switching to joint video/action denoising. The pair diagnostic tests the interaction boundary inside the same all-sites evaluation without changing the site-only estimand or falsification prediction.